

Universidad Autónoma de Chihuahua
Facultad de Ingeniería
Grupo 5K2

Proyecto

Bases de Datos Avanzadas

Kevin Andrés Rosales Rodríguez - 385814

Edwin Noe Zaragoza Alvarado - 385609

Manuel Ramirez Contreras- 385706

04/11/26

Objetivo

Implementar una base de datos distribuida con MongoDB sobre Docker utilizando un servidor de configuración, tres shards y un router mongos. La práctica extiende la arquitectura creada en la actividad anterior (actividad1-scott) incorporando: Despliegue de un clúster distribuido de 11 contenedores, utilización de la base de datos sample_mflix y la colección movies (21,349 documentos), configuración de la clave de partición (*Shard Key*) de tipo **Hashed** ({ id_movie: "hashed" }). Cada shard es un replica set de tres nodos. A cada película se le agrega el campo id_movie, se ejecutan las operaciones CRUD monitoreando cómo se distribuyen los documentos entre los shards y se demuestra el balanceo automático de chunks.

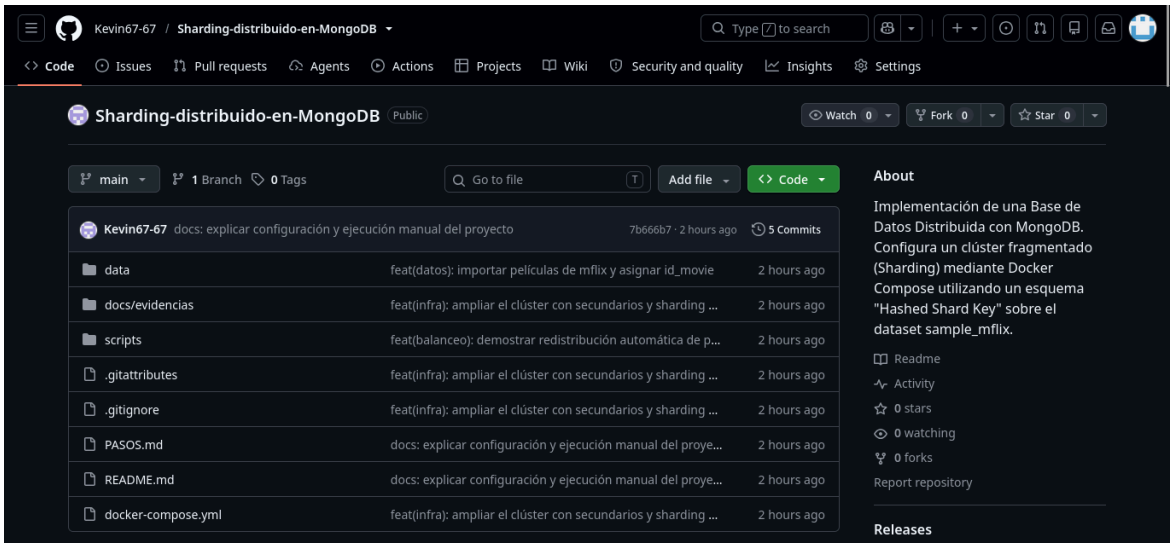
Introducción

El particionamiento horizontal o **sharding** es el método mediante el cual MongoDB distribuye grandes volúmenes de datos entre múltiples servidores físicos o instancias virtuales (*shards*). Cada fragmento actúa como una base de datos independiente que almacena un subconjunto de los datos del clúster.

En un entorno distribuido robusto, el sharding se complementa indispensablemente con la **replicación** (*Replica Sets*), lo que garantiza tolerancia a fallos, alta disponibilidad y redundancia de datos.

La elección de una clave de partición de tipo **Hashed** (`{ id_movie: "hashed" }`) calcula un valor hash MD5 de la clave, asegurando una distribución pseudoaleatoria y uniforme de los documentos a través de los shards, erradicando los cuellos de botella por inserciones secuenciales.

Repositorio



https://github.com/Kevin67-67/Sharding-distribuido-en-MongoDB

Evidencias (JSON): Se generan automaticamente al ejecutar los comandos
https://drive.google.com/drive/folders/1XnBnjt7Pbd1AYNHUICko0SGY5gcmGfHk?usp=sharing

Requisitos y archivos

Se utiliza Docker Desktop con motor de contenedores Linux, Docker Compose y PowerShell para ejecutar los comandos.

Archivo	Función
`docker-compose.yml`	Define los 11 contenedores, la red, los puertos, los volúmenes y el límite de caché de cada nodo
`scripts/lib/comun.js`	Guarda un snapshot de la distribución, los chunks y el balanceador en formato JSON
`scripts/01-replica-config.js`	Revisa el estado del replica set configRS
`scripts/02-replica-shard.js`	Agrega los nodos secundarios b y c a cada replica set de los shards
`scripts/03-configurar-sharding.js`	Habilita sharding en sample_mflix, crea el índice hashed y fragmenta movies
`scripts/04-importar-peliculas.js`	Descomprime el dataset, agrega id_movie e inserta las 21,349 películas

scripts/05-estado-replicas.js	Verifica qué nodo es PRIMARY y cuáles SECONDARY en cada réplica
scripts/crud/01-insertar.js	Inserta 3 películas de prueba
scripts/crud/02-consultar.js	Compara una consulta por id_movie contra una por género con explain()
scripts/crud/03-actualizar.js	Actualiza una película con updateOne
scripts/crud/04-eliminar.js	Elimina las 3 películas de prueba con deleteOne
scripts/06-preparar-balanceo.js	Pausa el balanceador y concentra todos los chunks en shard1RS
scripts/07-balanceo-automatico.js	Reactiva el balanceador y monitorea la migración de chunks
scripts/08-monitorear.js	Consulta rápida de la distribución actual en los shards
data/sample_mflix.movies.json.gz	Dataset comprimido con las 21,349 películas
PASOS.md	Explica la configuración inicial y la ejecución manual
README.md	Describe el proyecto y las especificaciones del clúster

El puerto publicado se cambió a 27030 porque 27017 ya estaba ocupado en Windows debido a anteriores prácticas. Además, cada nodo usa --wiredTigerCacheSizeGB 0.25 para limitar la memoria, por lo que los 11 contenedores ocupan en total unos 3 GB de RAM.

Arquitectura del clúster

El clúster se levanta con docker-compose.yml y reutiliza la red actividad1-scott_mongo-net de la actividad anterior, de modo que los contenedores se comunican por su nombre. Los 11 contenedores usan la imagen mongo:8.0.16.

El servidor de configuración (configsvr, replica set configRS) guarda los metadatos del clúster: los chunks, el catálogo de shards y el estado del balanceador. Cada shard es un replica set de tres nodos: shard1, shard1b y shard1c forman shard1RS; shard2, shard2b y shard2c forman shard2RS; y shard3, shard3b y shard3c forman shard3RS. En cada uno hay un nodo PRIMARY

que recibe las escrituras y dos SECONDARY que copian los cambios desde el oplog, lo que permite seguir funcionando si un nodo se cae.

mongos es el router al que se conectan los clientes. No guarda datos: consulta los metadatos del servidor de configuración y manda cada operación al shard que corresponde. Se accede a él desde el puerto 27030 del equipo.

Estructura de la colección movies

La colección movies de la base sample_mflix guarda información de películas: título, año, géneros, directores, reparto y calificaciones de IMDb y Rotten Tomatoes. MongoDB agrega _id a cada documento. Durante la importación se agrega además el campo id_movie, un número entero consecutivo del 1 al 21,349 que se usa como shard key.

Ejemplo de película:

```
{
  id_movie: 1, title: "Blacksmith Scene", year: 1893,
  genres: ["Short"], directors: ["William K.L. Dickson"],
  imdb: { rating: 6.2, votes: 1189, id: 5 }
}
```

La carga conserva los 21,349 documentos originales y solo agrega id_movie. Se hace en lotes de 500 documentos con writeConcern majority.

Elección de la shard key

Para la colección sample_mflix.movies elegimos { id_movie: "hashed" } como shard key porque id_movie es el número que identifica a cada película y permite buscarla directamente. Como es un número consecutivo, con una clave por rangos todas las inserciones nuevas caerían en el último chunk y un solo shard recibiría toda la carga. Con hash, las películas se reparten entre los shards aunque sus números sean consecutivos.

Decidimos no elegir genres ni year porque tienen pocos valores distintos y algunos se repiten mucho más que otros, como Drama. Eso formaría chunks que no se pueden dividir (jumbo chunks) y el balanceador no podría repartir los datos de forma pareja entre los shards.

Cuando se busca un valor exacto de id_movie, mongos calcula el hash y dirige la consulta a un solo shard. En cambio, una búsqueda solo por género necesita consultar los tres shards. Esta elección funciona para buscar películas por su número, entonces si la mayoría de las consultas fueran por género, tendríamos que considerar otra clave.

Aplicación y comandos

Primera configuración de servidores y replica sets

- `docker compose up -d configsvr shard1 shard1b shard1c shard2 shard2b shard2c shard3 shard3b shard3c`

Después de dejar tiempo para que MongoDB inicie:

- `docker compose exec -T configsvr mongosh --quiet --file /scripts/01-replica-config.js`
- `docker compose exec -T shard1 mongosh --quiet --file /scripts/02-replica-shard.js`
- `docker compose exec -T shard2 mongosh --quiet --file /scripts/02-replica-shard.js`
- `docker compose exec -T shard3 mongosh --quiet --file /scripts/02-replica-shard.js`

Router, registro de shards y fragmentación

- `docker compose up -d mongos`

Después del inicio del router:

- `docker compose exec -T mongos mongosh --quiet --file /scripts/03-configurar-sharding.js`

Carga de películas con id_movie

- `docker compose exec -T mongos mongosh --quiet --file /scripts/04-importar-peliculas.js`

La carga corresponde a la primera ejecución. No se repite al reiniciar.

Verificación de réplicas

- `docker compose exec -T mongos mongosh --quiet --file /scripts/05-estado-replicas.js`
- `docker compose exec -T mongos mongosh --quiet --file /scripts/08-monitorear.js`

Consultas

Desde PowerShell:

- `docker compose exec mongos mongosh`

Dentro de mongosh:

- `use sample_mflix`
- `sh.status()`
- `db.movies.countDocuments({})`
- `db.movies.getShardDistribution()`
- `db.getSiblingDB("admin").aggregate([{ $shardedDataDistribution: {} }, { $match: { ns: "sample_mflix.movies" } }]).toArray()`
- `exit`

Operaciones CRUD y monitoreo

Antes y después de cada operación se guarda un snapshot de la distribución de los documentos con $shardedDataDistribution, para ver en qué shard cambia el conteo. Los scripts se ejecutan desde mongos.

Inserción

Se insertaron tres películas de prueba con id_movie reservados: 9000001 (La última proyección), 9000002 (Órbita del desierto) y 9000003 (Cartas de noviembre). Al calcular el hash de cada id_movie, las películas cayeron en chunks de distintos shards, así que subió el conteo de los shards correspondientes sin alterar el balance general.

- `docker compose exec -T mongos mongosh --quiet --file /scripts/crud/01-insertar.js`

Consulta

Se compararon dos consultas con explain(). Al buscar por id_movie, mongos manda la consulta a un solo shard (plan SINGLE_SHARD). Al buscar por género, que no es la shard key, la manda a los tres shards y une los resultados (plan SHARD_MERGE).

- `docker compose exec -T mongos mongosh --quiet --file /scripts/crud/02-consultar.js`

Las consultas que se comparan son:

- `db.movies.find({ id_movie: 9000001 }).explain("executionStats")`
- `db.movies.find({ genres: "Drama" }).limit(5).explain("executionStats")`

Actualización

Se cambiaron el título, la duración y la calificación de IMDb de la película 9000001. Como el filtro incluye id_movie, la actualización va directo al shard que tiene el documento y se confirma con la mayoría de las réplicas (writeConcern majority).

- `docker compose exec -T mongos mongosh --quiet --file /scripts/crud/03-actualizar.js`

La operación que se ejecuta es:

- `db.movies.updateOne({ id_movie: 9000001, tipo_practica: "crud-mflix" }, { $set: { title: "La última proyección: edición restaurada", runtime: 110, "imdb.rating": 8.1 } }, { writeConcern: { w: "majority" } })`

Eliminación

Se eliminaron las tres películas de prueba una por una con deleteOne, guardando un snapshot antes y otro después de cada borrado. Al final el total volvió a 21,349 documentos, sin registros sobrantes.

- `docker compose exec -T mongos mongosh --quiet --file /scripts/crud/04-eliminar.js`

Balanceo de carga entre shards

Para ver el balanceador en acción primero se provocó un desbalanceo. Se pausó el balanceo de la colección con sh.disableBalancing y con moveChunk se movieron a shard1RS todos los chunks que estaban en shard2RS y shard3RS. Así los 21,349 documentos quedaron en shard1RS y los otros dos shards sin películas.

- `docker compose exec -T mongos mongosh --quiet --file /scripts/06-preparar-balanceo.js`

Después se reactivó el balanceo con chunkSize de 1 MB, para que los chunks fueran pequeños y se notaran las migraciones. El script tomó 60 muestras, una cada 10 segundos, y se registraron en config.changelog las operaciones moveChunk.commit. MongoDB pasó chunks de forma gradual a shard2RS y shard3RS hasta que la diferencia entre shards quedó por debajo del umbral de migración. Al terminar se restauró la configuración original de la colección.

- `docker compose exec -T mongos mongosh --quiet --file /scripts/07-balanceo-automatico.js`

Evidencias de ejecución y resultados

```
mozeu@O-O:~/Documentos/Bases de Datos/Sharding-distribuido-en-MongoDB$ docker compose up -d configsvr shard1 shard1b shard1c shard2 shard2b shard2c shard3 shard3b shard3c
[+] up 16/16
 ✓ Volume actividad1-scott_shard1b-data   Created                                                                    0.0s
 ✓ Volume actividad1-scott_shard1c-data   Created                                                                    0.0s
 ✓ Volume actividad1-scott_shard2b-data   Created                                                                    0.0s
 ✓ Volume actividad1-scott_shard2c-data   Created                                                                    0.0s
 ✓ Volume actividad1-scott_shard3b-data   Created                                                                    0.0s
 ✓ Volume actividad1-scott_shard3c-data   Created                                                                    0.0s
 ✓ Container actividad1-scott-configsvr-1 Started                                                                    1.8s
 ✓ Container actividad1-scott-shard1-1    Started                                                                    2.4s
 ✓ Container actividad1-scott-shard2-1    Started                                                                    2.8s
 ✓ Container actividad1-scott-shard3-1    Started                                                                    6.4s
 ✓ Container actividad1-scott-shard1b-1   Started                                                                    5.6s
 ✓ Container actividad1-scott-shard3b-1   Started                                                                    6.8s
 ✓ Container actividad1-scott-shard2c-1   Started                                                                    4.6s
 ✓ Container actividad1-scott-shard1c-1   Started                                                                    1.5s
 ✓ Container actividad1-scott-shard2b-1   Started                                                                    3.5s
 ✓ Container actividad1-scott-shard3c-1   Started                                                                    1.1s
```

Iniciar servidores

```
mozeu@O-O:~/Documentos/Bases de Datos/Sharding-distribuido-en-MongoDB$ docker compose exec -T configsvr mongosh --quiet --file /scripts/01-replica-config.js
EVIDENCIA: 1793842875890-configRS-b152da.json
{
  etapa: 'configRS',
  fecha: ISODate('2026-11-05T01:41:15.890Z'),
  estado: {
    set: 'configRS',
    date: ISODate('2026-11-05T01:41:15.888Z'),
    myState: 1,
    term: Long('2'),
    syncSourceHost: '',
    syncSourceId: -1,
    configsvr: true,
    heartbeatIntervalMillis: Long('2000'),
    majorityVoteCount: 1,
    writeMajorityCount: 1,
    votingMembersCount: 1,
    writableVotingMembersCount: 1,
    optimes: {
      lastCommittedOpTime: {
        ts: Timestamp({ t: 1793842875, i: 1 }),
        t: Long('2')
      },
```

```
mozeu@O-O:~/Documentos/Bases de Datos/Sharding-distribuido-en-MongoDB$ docker compose exec -T shard1 mongosh --quiet --file /scripts/02-replica-shard.js
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1793842897, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1793842897, i: 1 })
}
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1793842912, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1793842912, i: 1 })
}
```

```
mozeu@O-O:~/Documentos/Bases de Datos/Sharding-distribuido-en-MongoDB$ docker compose exec -T shard2 mongosh --quiet --file /scripts/02-replica-shard.js
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1793842943, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1793842943, i: 1 })
}
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1793842958, i: 2 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1793842958, i: 1 })
}
```

```
mozeu@O-O:~/Documentos/Bases de Datos/Sharding-distribuido-en-MongoDB$ docker compose exec -T shard3 mongosh --quiet --file /scripts/02-replica-shard.js
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1793842976, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1793842976, i: 1 })
}
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1793842991, i: 2 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1793842991, i: 1 })
}
```

Replica sets

```
mozeu@O-O:~/Documentos/Bases de Datos/Sharding-distribuido-en-MongoDB$ docker compose up -d mongos
[+] up 1/1
 ✓ Container actividad1-scott-mongos-1 Started                                                                                                0.6s
mozeu@O-O:~/Documentos/Bases de Datos/Sharding-distribuido-en-MongoDB$ docker compose exec -T mongos mongosh --quiet --file /scripts/03-configurar-sharding.js
{
  shards: [
    {
      _id: 'shard1RS',
      host: 'shard1RS/shard1:27017,shard1b:27017,shard1c:27017',
      state: 1,
      topologyTime: Timestamp({ t: 1793248816, i: 10 }),
      replSetConfigVersion: Long('5')
    },
    {
      _id: 'shard2RS',
      host: 'shard2RS/shard2:27017,shard2b:27017,shard2c:27017',
      state: 1,
      topologyTime: Timestamp({ t: 1793248816, i: 28 }),
      replSetConfigVersion: Long('5')
    },
    {
      _id: 'shard3RS',
      host: 'shard3RS/shard3:27017,shard3b:27017,shard3c:27017',
      state: 1,
      topologyTime: Timestamp({ t: 1793248816, i: 51 }),
      replSetConfigVersion: Long('5')
    }
  ],
```

Router y sharding

```
mozeu@O-O:~/Documentos/Bases de Datos/Sharding-distribuido-en-MongoDB$ docker compose exec -T mongos mongosh --quiet --file /scripts/04-importar-peliculas.js
{
  lote: 1,
  insertados: 500,
  acknowledged: true
}
{
  lote: 2,
  insertados: 500,
  acknowledged: true
}
{
  lote: 3,
  insertados: 500,
  acknowledged: true
}
{
  lote: 4,
  insertados: 500,
  acknowledged: true
}
{
  lote: 5,
  insertados: 500,
  acknowledged: true
}
{
  lote: 6,
  insertados: 500,
  acknowledged: true
}
```

Carga de películas con id_movie

```
mozeu@O-O:~/Documentos/Bases de Datos/Sharding-distribuido-en-MongoDB$ docker compose exec -T mongos mongosh --quiet --file /scripts/05-estado-replicas.js
EVIDENCIA: 1793843266679-replicacion-435e7c.json
{
  etapa: 'replicacion',
  fecha: ISODate('2026-11-05T01:47:46.679Z'),
  estados: [
    {
      replicaSet: 'shard1RS',
      primaryTime: ISODate('2026-11-05T01:47:38.000Z'),
      miembros: [
        {
          host: 'shard1:27017',
          estado: 'PRIMARY',
          optimeDate: ISODate('2026-11-05T01:47:38.000Z'),
          muestra: []
        },
        {
          host: 'shard1b:27017',
          estado: 'SECONDARY',
          optimeDate: ISODate('2026-11-05T01:47:38.000Z'),
          muestra: []
        },
        {
          host: 'shard1c:27017',
          estado: 'SECONDARY',
          optimeDate: ISODate('2026-11-05T01:47:38.000Z'),
          muestra: []
        }
      ],
```

Estado de réplicas

```
EVIDENCIA: 1793843294657-crud-insercion-c42ae7.json
{
  etapa: 'crud-insercion',
  fecha: ISODate('2026-11-05T01:48:14.657Z'),
  operacion: 'insertMany',
  resultado: InsertManyResult {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('6a000000000000000000001'),
      '1': ObjectId('6a000000000000000000002'),
      '2': ObjectId('6a000000000000000000003')
    }
  },
```

CRUD: inserción

```
EVIDENCIA: 1793843309420-crud-consulta-e5b226.json
{
  etapa: 'crud-consulta',
  fecha: ISODate('2026-11-05T01:48:29.420Z'),
  operacion: 'find',
  consulta: {
    id_movie: 9000001
  },
  documentos: [
    {
      _id: ObjectId('6a000000000000000000001'),
      id_movie: 9000001,
      title: 'La última proyección',
      year: 2026,
      genres: [
        'Drama'
      ],
      runtime: 104,
      languages: [
        'Spanish'
      ],
      countries: [
        'Mexico'
      ],
      directors: [
        'Elena Navarro'
      ],
      plot: 'Una proyeccionista recupera el cine de su barrio.',
      tipo_practica: 'crud-mflix'
    }
  ],
```

CRUD: consulta

```
EVIDENCIA: 1793843320310-crud-actualizacion-34087d.json
{
  etapa: 'crud-actualizacion',
  fecha: ISODate('2026-11-05T01:48:40.310Z'),
  operacion: 'updateOne',
  filtro: {
    id_movie: 9000001,
    tipo_practica: 'crud-mflix'
  },
  cambio: {
    '$set': {
      title: 'La última proyección: edición restaurada',
      runtime: 110,
      'imdb.rating': 8.1
    }
  },
  resultado: UpdateResult {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  },
  lecturaPosterior: {
    _id: ObjectId('6a000000000000000000001'),
    id_movie: 9000001,
    title: 'La última proyección: edición restaurada',
    year: 2026,
    genres: [
      'Drama'
    ],
    runtime: 110,
    languages: [
      'Spanish'
    ],
```

CRUD: actualización

```
EVIDENCIA: 1793843345339-balanceo-configuracion-previa-b0ccaa.json
{
  etapa: 'balanceo-configuracion-previa',
  fecha: ISODate('2026-11-05T01:49:05.339Z'),
  uuid: UUID('7ee65beb-dc45-4bdd-8cb6-5ef0e3d5e354'),
  configuracionAnterior: {
    chunkSize: 128,
    enableAutoMerger: true,
    enableBalancing: true
  }
}
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1793843345, i: 7 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1793843345, i: 7 })
}
{
  millis: 2231,
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1793843352, i: 4336 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1793843352, i: 4336 })
}
```

Balanceo: desbalanceo forzado

```
  balance: {
    chunkSize: 128,
    balancerCompliant: true,
    ok: 1,
    '$clusterTime': {
      clusterTime: Timestamp({ t: 1793843368, i: 2 }),
      signature: {
        hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
        keyId: Long('0')
      }
    },
    operationTime: Timestamp({ t: 1793843368, i: 2 })
  }
}
```

Balanceo: migración automática de chunks

```
    distribucion: [
      {
        ns: 'sample_mflix.movies',
        shards: [
          {
            shardName: 'shard1RS',
            numOrphanedDocs: 13109,
            numOwnedDocuments: 8240,
            ownedSizeBytes: 13274640,
            orphanedSizeBytes: 21118599
          },
          {
            shardName: 'shard3RS',
            numOrphanedDocs: 0,
            numOwnedDocuments: 6490,
            ownedSizeBytes: 10533270,
            orphanedSizeBytes: 0
          },
          {
            shardName: 'shard2RS',
            numOrphanedDocs: 0,
            numOwnedDocuments: 6619,
            ownedSizeBytes: 10623495,
            orphanedSizeBytes: 0
          }
        ]
      }
    ],
```

Distribución final de shards

Conclusión

En esta práctica se extendió el clúster MongoDB de la actividad anterior a 11 contenedores: un servidor de configuración, tres shards con replica sets de tres nodos cada uno y un router mongos. Se cargó la colección movies de sample_mflix con sus 21,349 documentos, a los que se les agregó el campo id_movie, y se fragmentó con la shard key { id_movie: "hashed" }. Se eligió hash porque id_movie es un número consecutivo y con una clave por rangos todas las inserciones nuevas habrían llegado a un solo shard.

La distribución quedó en 7,116, 7,117 y 7,116 documentos en shard1RS, shard2RS y shard3RS, con un total de 21,349, es decir, casi perfectamente pareja. Las operaciones CRUD funcionaron desde mongos y con explain() se comprobó que una consulta por id_movie va a un solo shard, mientras que una consulta por género se envía a los tres. Por último, después de concentrar todos los chunks en shard1RS, el balanceador volvió a repartirlos entre los shards sin perder datos ni detener el servicio.

Referencias

MongoDB. *Deploy a Self-Managed Sharded Cluster*. https://www.mongodb.com/docs/manual/tutorial/deploy-shard-cluster/

MongoDB. *Hashed Sharding — MongoDB 8.0*. https://www.mongodb.com/docs/v8.0/core/hashed-sharding/

MongoDB. *shardCollection — MongoDB 8.0*. https://www.mongodb.com/docs/v8.0/reference/command/shardcollection/

MongoDB. *$shardedDataDistribution*. https://www.mongodb.com/docs/manual/reference/operator/aggregation/shardeddatadistribution/

MongoDB. *Manage Sharded Cluster Balancer*. https://www.mongodb.com/docs/manual/tutorial/manage-sharded-cluster-balancer/

Docker. *Docker Compose*. https://docs.docker.com/compose/

Kavaturu, U. *MongoDB Sharding Demo with Docker*. https://github.com/udaykirankavaturu/sharding-demo

Material de referencia de la actividad: BDA_SettingUpMongoDBCluster.pdf y BDA_MongoDBSharding.pdf.